Computational Study of Algorithms for the Parametric Maximum Closure Problem on Directed Forests

Technical Report

Valerio Dose Fabio Furini Marco Locatelli

September 2026

Abstract

This is a companion technical report to the manuscript *"On parametric Maximum Closure Problems over precedence forests"* (Dose, Furini and Locatelli). The manuscript introduces the algorithms studied here (`PaC`, `DPaC`, `HPaC`, `DHPaC`, `HIPaC`, `HOPaC` and `RaC`) with their correctness proofs and complexity analysis, and reports a compact selection of the experiments. This report contains the full computational study: instance design, validation methodology, measurement protocol, campaign definitions, and complete results broken down *by size, by density and by coefficient family* – together with the implementation analysis and the paired-ratio dispersion that the manuscript has no room for. Every number below is generated from the raw benchmark data of release `v0.3.0` by the repository's own pipeline; none is hand-typed.

1 Scope and relationship to the manuscript

The manuscript is the reference for the problem, the algorithms and their analysis. This report is the reference for *how* the experiments were run and *what exactly* was measured, and it reports the breakdowns that the manuscript summarizes in one sentence or omits. Section 2 recaps problem and algorithms; Sections 3 to 6 document instances, correctness, protocol and campaign design; Section 7 reports every result.

The whole study was rerun from scratch in a single sweep on 2026-08-31/09-01 from one frozen commit, under the preregistered plan `docs/EXPERIMENTAL_PLAN_V3.md`. All earlier benchmark results are superseded and are cited nowhere in this report.

2 Problem and algorithms recap

Let $G = (V, A)$ be a directed acyclic graph on n vertices, where arc $(i, j) \in A$ means that including vertex i forces the inclusion of vertex j. Each vertex has an integer profit p_i (possibly negative) and a strictly positive integer weight w_i. For a closed subset S, let $P(S) = \sum_{i \in S} p_i$ and $W(S) = \sum_{i \in S} w_i$. The parametric Maximum Closure Problem asks for the set of optimal solutions of $u(\lambda) := \max_{S \text{ closed}} P(S) - \lambda W(S)$ as a function of λ. Function u is convex piecewise linear with at most n breakpoints; the optimal solutions at consecutive breakpoints are nested, which partitions V into k *closure layers* $\mathcal{I}_1, \ldots, \mathcal{I}_k$, strictly decreasing in profit-to-weight ratio. Computing that ordered partition is the task studied here, on directed forests (underlying undirected graph a forest, orientations arbitrary).

`PaC` Peel-and-Contract, direct-scan reference algorithm: $\mathcal{O}(n^2)$ time, $\mathcal{O}(n)$ space. Repeatedly peels the current best final vertices or contracts an arc, selecting the maximum-ratio candidate by a linear scan.

`DPaC` dual of `PaC`: layers in increasing ratio order, same complexity.

`HPaC` heap-based variant: the scans are replaced by two candidate heaps. $\mathcal{O}(n^2 \log n)$ worst case, $\mathcal{O}(n)$ space.

`DHPaC` dual of `HPaC`.

`HIPaC/HOPaC` specialized heap variants for forests of in-trees (out-degree ≤ 1) or out-trees (in-degree ≤ 1): $\mathcal{O}(n \log n)$ time.

`RaC` Rake-and-Compress, top-tree algorithm: $\mathcal{O}(n \log n)$ time and space, any orientation.

`BPPF` Bounded-Precision Parametric Pseudoflow, unmodified upstream implementation for general precedence graphs; external baseline (Section 7.6).

All algorithms of this repository return the same canonical object – the ordered sequence of closure layers with exact rational thresholds – computed with exact integer arithmetic: 64-bit integer coefficients, 128-bit cross-multiplications for every ratio comparison, no floating point in any decision path.

3 Instance generation

Instances are stored in a capacity-free text format (`.pcf`): the arc list of a forest on n vertices, one integer profit and one strictly positive integer weight per vertex, arc (u, v) encoding $x_u \leq x_v$. Every instance is produced by a seeded deterministic generator – never hand-edited – and is determined by a topology, a coefficient family, a size and a seed. Each is manifested with its SHA-256 checksum, topology classification and coefficient bounds (`instances/manifests/`), so any regenerated or downloaded archive is verifiable byte-for-byte.

Affine-coefficient families. Six families, chosen for the shape of the ratios p_i/w_i and never after an external classification. `independent-positive`: w_i, p_i independent uniform on $\{1, \ldots, 1000\}$. `independent-signed`: as above with p_i uniform on $\{-1000, \ldots, 1000\}$. `correlated`: $p_i = w_i + \delta_i$, δ_i uniform on $\{-50, \ldots, 50\}$. `anti-correlated`: $p_i = (1001 - w_i) + \delta_i$. `near-ties`: one target ratio num/den per instance (num, den uniform on $\{1, \ldots, 20\}$), then t_i uniform on $\{1, \ldots, 50\}$, $w_i = \text{den} \cdot t_i$, $p_i = \text{num} \cdot t_i + \jmath_i$ with jitter $\jmath_i$ uniform on $\{-2, -1, 1, 2\}$. `exact-ties`: $g = \max(1, \lfloor n/20 \rfloor)$ groups with their own exact ratio, vertex i assigned to group $i \bmod g$, so vertices sharing a threshold are spread over the graph rather than adjacent.

The last two families are the ones that stress exact arithmetic: they place thresholds at infinitesimal distance (`near-ties`) or exactly equal (`exact-ties`). They also produce markedly fewer closure layers, which is the single most useful fact for reading absolute times, since the work of every algorithm scales with the number of breakpoints (Table 1): at $n \geq 10\,000$ the median count ranges from 1 222 (`near-ties`) to 33 183 (`independent-signed`), a factor of 27 between families of the same size.

Table 1: Median number of closure layers per coefficient family, i.e. how much parametric structure each family creates. This drives the running time of every algorithm and, in Section 7.6, the number of probes handed to `BPPF`.

	median # closure layers	
family	$n \leq 1\,000$	$n \geq 10\,000$
`anti-correlated`	342	30379
`correlated`	337	24825
`exact-ties`	105	7884
`independent-positive`	344	32374
`independent-signed`	346	33183
`near-ties`	164	1222

Topologies. `mixed-forest`: each vertex $v \in \{2, \ldots, n\}$ is connected to a uniformly chosen predecessor $u < v$ independently with probability ϱ, and each edge is oriented (u, v) or (v, u) with probability $1/2$. With $\varrho \in \{0.3, 0.6, 0.9, 1.0\}$ the forest has $\varrho(n-1)$ arcs and $n - \varrho(n-1)$ components in expectation, so $\varrho = 1.0$ gives a spanning tree and $\varrho = 0.3$ a forest of many small trees. `in-forest`: vertices scanned $u \in \{1, \ldots, n-1\}$, with probability ϱ one arc (u, j) with j uniform on $\{u+1, \ldots, n\}$, giving out-degree at most one; `out-forest` symmetric. `path-mixed`, `binary-mixed`, `star-mixed`: fixed underlying shape (path; balanced binary tree with parent $\lfloor (v-1)/2 \rfloor$; star with hub 0), only orientations randomized.

4 Validation methodology

Exhaustive enumeration oracle. The CTest suite checks every algorithm against an oracle that enumerates all closed sets, builds their affine lines $P - \lambda W$ and computes the exact upper envelope – a method structurally unrelated to any algorithm under test. Coverage: every directed forest with at most four vertices over a finite coefficient grid, exhaustively; 10 000 random forests up to 11 vertices; mixed-orientation path, binary and star trees up to 257 vertices; 6 000 in-forest and 6 000 out-forest instances plus 2 000 larger ones per orientation; and the structural invariants (the layers partition V, each is a closure increment, thresholds strictly decreasing).

Cross-algorithm differential agreement. Every campaign runs several algorithms on the same instance and compares the returned sequences – partition, thresholds and canonical order, not merely the layer count – via a 64-bit fingerprint of the canonical serialization. Disagreements are listed in `results/processed/mismatches.csv`, never averaged away. Table 2 reports the outcome: over 16 200 instances and 292 080 timed runs, every pair of algorithms benchmarked on the same instance returned the identical sequence.

Sanitizers, CI and exactness. The suite is built and run on every push in three configurations: GCC Release, GCC Debug with the address and undefined-behaviour sanitizers, and Clang Debug. All three pass at the frozen commit (`results/TEST_REPORT_2026-08-31.md`). `validate_instance` rejects any instance whose total absolute profit or total weight would leave the exact-arithmetic safety margin (`INT64_MAX`/4), so overflow is refused explicitly rather than silently wrapped.

Table 2: Instances benchmarked and cross-algorithm disagreements per campaign. Campaign G (Section 7.6) is reported separately, `BPPF` being an external baseline rather than one of our algorithms.

Campaign	Instances	Mismatches
campaign_b	2400	0
campaign_c	2400	0
campaign_d_binary	600	0
campaign_d_path	600	0
campaign_d_star	600	0
campaign_e_in	4800	0
campaign_e_out	4800	0

5 Experimental protocol

All algorithms are implemented in C++ and run single-threaded on a Linux machine (Ubuntu 22.04.5, kernel 6.8) with an Intel Core i7-12700 and 32 GB of RAM, GCC 11.4.0 at `-O3`. Every benchmark process is pinned to one physical core with `taskset`; the six campaign lanes ran concurrently on distinct cores, never two processes on one core. Only the algorithm call is timed: parsing, result hashing and correctness comparison are outside the timed region. Algorithm order is shuffled independently per repetition. Repetitions: 11 on the medium random campaign, 3 on the larger and structured campaigns and on the `BPPF` comparison, where a single run takes from tens of milliseconds to tens of seconds. A 300 s wall-clock timeout and an 8 GiB memory ceiling (`ulimit -v`) applied to every run; *no run of the study hit either*, so no observation is censored.

What we measure, and what "peak RSS" means. Time is wall-clock time of the algorithm call (`std::chrono::steady_clock`); on an otherwise idle single-threaded run this tracks CPU time closely. Memory is reported as *peak RSS*, the peak *resident set size* of the process: the maximum amount of physical RAM the process had mapped at any instant, excluding pages swapped out or never touched. It is read from `getrusage(RUSAGE_SELF).ru_maxrss` after each repetition. Two properties of this counter matter when reading memory figures. First, it is a *process-lifetime* peak and never decreases, so within one invocation that benchmarks several algorithms the value attributed to the second algorithm includes the peak of the first: memory numbers are therefore only meaningful for single-algorithm invocations, and this report quotes them only from such runs (Section 7.2). Second, it counts pages actually resident, so it includes the allocator's own overhead and is the figure that determines whether a run survives a memory ceiling – which is exactly what we want to report.

Statistics. Absolute times are medians over repetitions and then over instances. Comparisons are always *paired*: the ratio is computed per instance, on the same instance solved by both algorithms, and we report the median with the interquartile range (IQR) of those per-instance ratios – never a ratio of aggregate means. The median is used deliberately: it is invariant under inversion of the comparison, unlike the arithmetic mean of ratios, which depends on which algorithm sits in the denominator; it is insensitive to the right tail of the timing noise, which is asymmetric by construction, since system interference can only slow a run down and never speed it up; and on every comparison where an advantage is claimed it is the conservative choice. As a check, on the `BPPF` comparison at $n = 1\,000$ the three aggregates are 4.5 (median), 5.8 (geometric mean) and 8.7 (arithmetic mean): the reported figure is the smallest. Where a population mixes regimes – as the large random campaign does across four densities – no single aggregate describes it, and the breakdowns of Section 7.3 are reported instead of relying on the overall figure.

Table 3: The campaigns of the sweep. Two preregistered size cutoffs apply, both because the asymptotic trend is established below them and running further would consume hours without adding information: `PaC`/`DPaC` stop at $n = 20\,000$ on random forests, and `HPaC`/`DHPaC` stop at $n = 20\,000$ on the star class, where `PaC` is cheap and is run on the full range instead.

Campaign	Topology	Sizes n	Algorithms
A – correctness	all six	≤ 11 (exhaustive at 4)	all, vs. enumeration oracle
B – medium random	`mixed-forest`	$100, 200, \ldots, 1\,000$	`PaC`, `DPaC`, `HPaC`, `DHPaC`, `RaC`
C – large random	`mixed-forest`	$10\,000, \ldots, 100\,000$	`HPaC`, `DHPaC`, `RaC`; `PaC`/`DPaC` at $10^4, 2{\cdot}10^4$
D – structured	`path/binary/star-mixed`	$100, \ldots, 100\,000$ (10 sizes)	`PaC`, `DPaC`, `HPaC`, `DHPaC`, `RaC`
E – orientations	`in-`/`out-forest`	$100, \ldots, 100\,000$ (20 sizes)	`HPaC`, `DHPaC`, `HIPaC`/`HOPaC`, `RaC`
G – `BPPF` baseline	`mixed-forest`	$100, 200, \ldots, 1\,000$	`HPaC` vs. `BPPF`

Known limitation. The machine has no passwordless root, so the CPU governor stayed at `powersave` rather than `performance`. Absolute numbers therefore carry ordinary frequency-scaling noise, mitigated (not eliminated) by median-of-repetitions reporting, core pinning, and by expressing every headline comparison as a paired same-instance ratio: a frequency shift during one instance's measurement affects both algorithms compared on it almost identically.

6 Campaign design

For every topology and size, 240 instances are generated (four densities, six coefficient families, ten seeds), or 60 for the structured families, which have no density parameter. `mixed-forest`, `in-forest` and `out-forest` instances exist for $n \in \{100, 200, \ldots, 1\,000\} \cup \{10\,000, 20\,000, \ldots, 100\,000\}$; the structured families for $n \in \{100, 200, 500, 1\,000, 2\,000, 5\,000, 10\,000, 20\,000, 50\,000, 100\,000\}$.

7 Results

7.1 Direct scan versus heap on random forests

Figure 1 reports every algorithm of campaign B on the same instances. The two families are indistinguishable up to $n \approx 300$ and the heap pays off from $n \approx 400$ on: the median paired ratio `PaC`/`HPaC` is 0.91 at $n = 100$ – the direct scan is marginally faster where heap maintenance does not yet pay for itself – crosses 1 around $n = 400$, and reaches 2.3 at $n = 1\,000$, 20.3 at $n = 10\,000$ and 42.9 at $n = 20\,000$. The duals track their primals throughout (`DPaC`/`DHPaC` reaches 54.6 at $n = 20\,000$). The mechanism is the one anticipated by the manuscript: `PaC` re-scans every candidate ratio at every iteration, while `HPaC` extracts the maximum in logarithmic time and refreshes only the candidates touched by the last peel or contraction.

The aggregate hides two systematic effects, which the breakdowns make visible. By density (Table 4), all algorithms slow down as the forest becomes more connected, but not equally: from $\varrho = 0.3$ to $\varrho = 1.0$ `HPaC` grows by a factor of about 2 while `RaC` grows by less, which is the medium-size beginning of the density effect that dominates Section 7.3. By family (Table 5), the two tie-stressing families are the fastest for every algorithm, consistently with their smaller number of closure layers (Table 1); the ordering between algorithms is the same in every family, so no family reverses a conclusion.

	CPU time (ms)				
n	PaC	DPaC	HPaC	DHPaC	RaC
100	0.1	0.1	0.1	0.1	0.3
200	0.2	0.2	0.2	0.2	0.6
300	0.3	0.3	0.3	0.3	0.9
400	0.5	0.5	0.3	0.3	1.2
500	0.7	0.7	0.4	0.4	1.4
600	0.9	1.0	0.5	0.5	1.8
700	1.2	1.3	0.6	0.6	2.0
800	1.8	1.6	0.8	0.7	2.7
900	2.2	2.0	0.9	0.8	3.3
1 000	2.2	2.4	0.9	0.9	3.0

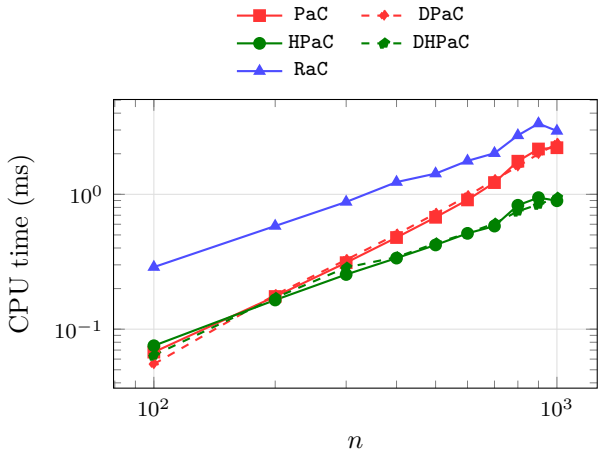

Figure 1: Median CPU time of all five algorithms on `mixed-forest` instances with $n \in \{100, 200, \ldots, 1\,000\}$ (campaign B, 240 instances per size).

Table 4: Campaign B by density.

	median CPU time (ms)		
ϱ	PaC	HPaC	RaC
0.3	0.6	0.2	0.6
0.6	0.7	0.3	1.2
0.9	0.8	0.7	2.1
1.0	0.9	0.9	2.3

Table 5: Campaign B by coefficient family.

	median CPU time (ms)		
family	PaC	HPaC	RaC
`anti-correlated`	0.8	0.4	1.2
`correlated`	0.8	0.4	1.2
`exact-ties`	0.5	0.3	1.2
`independent-positive`	0.8	0.4	1.3
`independent-signed`	0.8	0.4	1.2
`near-ties`	0.6	0.3	1.2

7.2 Heap policy, and why the implementation attains the $\mathcal{O}(n)$ space bound

The candidate heaps admit two natural implementations. The choice is invisible in the asymptotic time bound and decisive in practice. A push-only *lazy-deletion* heap re-pushes one entry per incident arc whenever a vertex's closure sum changes, and discards stale entries only when they surface at the top: its size is bounded by the number of touch operations, not by n. On a star this is pathological – every peel or contraction touches the hub, and each touch re-pushes one entry per incident arc, so the heap accumulates $\Theta(n^2)$ entries. The implementation used throughout this study instead rebuilds each heap from the live candidates whenever its size exceeds a constant multiple of the number of live candidates: an amortized $\mathcal{O}(\log n)$ cost per operation that leaves the worst-case time bound unchanged and makes the memory of the implementation $\mathcal{O}(n)$ on *every* input, matching the space bound proved in the manuscript.

Table 8 quantifies the difference on stars, with single-algorithm, single-repetition invocations (the only regime in which peak RSS is attributable to one algorithm, see Section 5). The lazy heap's peak resident memory grows by a factor of 100 when n grows by a factor of 10 – quadratic, as predicted – and reaches 1.5 GiB at $n = 10^4$, where the rebuild policy stays at 8 MiB; extrapolating, the lazy policy exhausts an 8 GiB ceiling by $n \approx 20\,000$. The rebuild policy is also *faster*, by about 4× here and about 2× on large random forests, with bit-identical output on every instance tested. This is why the study reports a single `HPaC`: the rebuild policy dominates the lazy one on both axes, and the same policy is used in `DHPaC`.

Table 6: Campaign B, paired ratio PaC/HPaC by family: values above 1 favour HPaC.

family	#inst	median PaC/HPaC	IQR
anti-correlated	400	1.706	1.804
correlated	400	1.693	1.753
exact-ties	400	1.220	0.747
independent-positive	400	1.716	1.863
independent-signed	400	1.675	1.818
near-ties	400	1.397	1.015

Table 7: Campaign B, paired ratio RaC/HPaC by family.

family	#inst	median RaC/HPaC	IQR
anti-correlated	400	3.117	0.674
correlated	400	3.173	0.684
exact-ties	400	3.300	0.724
independent-positive	400	3.187	0.633
independent-signed	400	3.183	0.636
near-ties	400	3.166	0.723

Table 8: The two heap implementations on star-mixed instances (independent-positive, seed 0, one repetition, Release build). For reference, PaC, which maintains no candidate structure at all, peaks at 5.6 MiB at $n = 10^4$.

	lazy-deletion heap		rebuild policy (used here)	
n	time (ms)	peak RSS (MiB)	time (ms)	peak RSS (MiB)
1 000	139.0	15.6	64.1	3.9
2 000	628.7	51.5	233.1	4.0
5 000	5 440.9	388.0	1 422.1	6.4
10 000	27 020.7	1 541.2	6 396.5	8.2

7.3 Large random forests: size, density and family

On mixed-forest instances up to $n = 10^5$ (Fig. 2), HPaC computes the full parametric solution in about 0.2 s and DHPaC stays within 25% of it at every size. The median paired ratio RaC/HPaC grows from 4.2 at $n = 10^4$ to 13.3 at $n = 10^5$, with a widening IQR: RaC's relative disadvantage on these instances becomes both larger and more variable as the forests grow.

That aggregate mixes four qualitatively different regimes. Tables 9 and 10 break it down by density: the median RaC/HPaC ratio is 12.9 at $\varrho = 0.3$, 17.0 at $\varrho = 0.6$, 5.0 at $\varrho = 0.9$ and only 2.0 at $\varrho = 1.0$, i.e. on single spanning trees, and the IQR shrinks in the same direction ($10.2 \to 0.4$), so the sparse regime is not only worse for RaC but far more variable. The intuition is structural: a sparse forest splits into many small components, on which HPaC's contractions are almost free, while RaC still pays its cluster bookkeeping per component; on one spanning tree the two are closest. Note that the ordering never reverses – HPaC is faster at every density on these instances – but the margin varies by almost an order of magnitude, which is why a single aggregate would be misleading.

The breakdown by family (Tables 11 and 12) shows the opposite picture: it is remarkably flat. Absolute times vary by at most 14% across families for each algorithm, and the paired ratio RaC/HPaC stays between 5.80 and 6.11 in all six. The coefficient family therefore changes *how much parametric structure* there is (Table 1) but not *which algorithm wins*: on these instances the topology, not the coefficients, decides.

7.4 Structured instance classes

Paths and balanced binary trees behave like random forests (Figs. 3 and 4): the median ratio RaC/HPaC lies between 5.8 and 9.0 on paths and between 3.2 and 3.9 on binary trees, from $n = 100$ to $n = 100\,000$, with no trend towards a crossover. On both classes PaC behaves as its $\mathcal{O}(n^2)$ bound predicts and is left far behind by $n = 10^4$.

Stars invert the picture, increasingly sharply with n (Fig. 5): RaC is already faster than HPaC at $n = 100$ (median ratio 0.75) and about 250 times faster at $n = 20\,000$. The mechanism is the

	CPU time (ms)		
n	HPaC	DHPaC	RaC
10 000	12.7	13.2	50.5
20 000	28.0	29.8	131.5
30 000	44.9	47.2	262.8
40 000	62.9	66.9	399.7
50 000	84.3	92.3	543.9
60 000	96.1	117.9	652.5
70 000	117.4	141.7	837.0
80 000	141.1	161.5	1 027.5
90 000	164.9	198.1	1 304.7
100 000	203.9	214.6	1 709.4

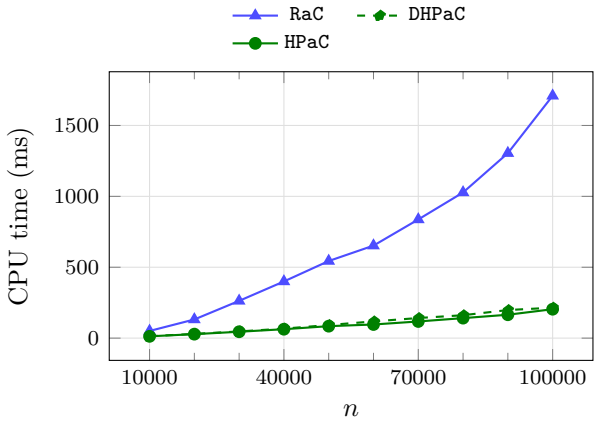

Figure 2: Median CPU time on `mixed-forest` instances with $n \in \{10\,000, 20\,000, \ldots, 100\,000\}$ (campaign C, 240 instances per size).

Table 9: Campaign C absolute times by density.

	median CPU time (ms)		
ϱ	HPaC	DHPaC	RaC
0.3	27.8	34.4	349.2
0.6	52.4	62.3	902.1
0.9	140.7	159.3	709.4
1.0	221.4	261.4	432.7

Table 10: Campaign C, paired ratio RaC/HPaC by density.

rho	Paired instances	Median rac/hpac	IQR
0.3	600	12.924	10.236
0.6	600	17.029	12.518
0.9	600	4.972	1.893
1.0	600	2.004	0.359

one of Section 7.2 seen from the algorithmic side: every peel or contraction touches the hub and invalidates the candidate ratios of a constant fraction of the remaining leaves, so *any* heap-based schedule pays $\Theta(n)$ maintenance per event, while RaC's rake operations absorb all leaves in $\mathcal{O}(\log n)$ rounds regardless of the hub's update history. The direct scan is far less affected, since it maintains no candidate structure: PaC stays about one order of magnitude below HPaC over the whole range and is overtaken by RaC only between $n = 1\,000$ and $n = 2\,000$. This is the one class of the study where the better worst-case bound of RaC is visible in practice, and there it is unambiguous. The effect is uniform across coefficient families (Table 13): the median RaC/HPaC ratio lies between 0.060 and 0.064 in all six, so it is the topology alone that produces it.

7.5 Specialized variants for single orientations

On the orientation each is built for, the specialized $\mathcal{O}(n \log n)$ variants take a stable fraction of the general algorithm's time (Figs. 6 and 7): the median ratio HIPaC/HPaC lies between 0.52 and 0.75 on in-forests and HOPaC/HPaC between 0.57 and 0.78 on out-forests, with no erosion up to $n = 10^5$ and no family in which the advantage disappears (Tables 15 and 16). The gain reflects the structural reason given in the manuscript: on a single orientation the minimal preceding (respectively succeeding) set of every arc collapses to a singleton, so one heap value per vertex suffices and no closure-sum propagation is needed. On the same instances RaC loses ground as n grows, from about 2.8× HPaC at $n \le 1\,000$ to 12.8× at $n = 10^5$, mirroring the trend on general mixed forests.

We record explicitly that this advantage is smaller than the one reported in the first version of the manuscript, where the specialized variants were 4–10 times faster than the general algorithm. The direction of the change is expected – the general HPaC is substantially faster than it was, for

Table 11: Campaign C absolute times by family.

family	median CPU time (ms) HPaC	DHPaC	RaC
anti-correlated	68.5	78.2	563.3
correlated	67.2	75.6	543.8
exact-ties	63.0	71.2	505.1
independent-positive	70.7	79.2	559.9
independent-signed	70.2	77.8	552.1
near-ties	62.0	68.0	490.3

Table 12: Campaign C, paired ratio RaC/HPaC by family.

family	#inst	median RaC/HPaC	IQR
anti-correlated	400	6.002	11.174
correlated	400	6.023	11.491
exact-ties	400	5.983	13.354
independent-positive	400	5.796	11.053
independent-signed	400	5.904	10.993
near-ties	400	6.110	13.377

	CPU time (ms)			
n	PaC	HPaC	DHPaC	RaC
100	0.1	0.1	0.1	0.5
200	0.2	0.2	0.2	1.1
500	0.8	0.4	0.4	2.9
1 000	2.9	0.8	0.9	6.1
2 000	10.5	1.7	1.8	12.9
5 000	–	4.4	4.7	37.9
10 000	–	9.0	9.7	80.5
20 000	–	19.2	20.9	175.3
50 000	–	54.9	60.6	475.8
100 000	–	117.6	134.0	926.2

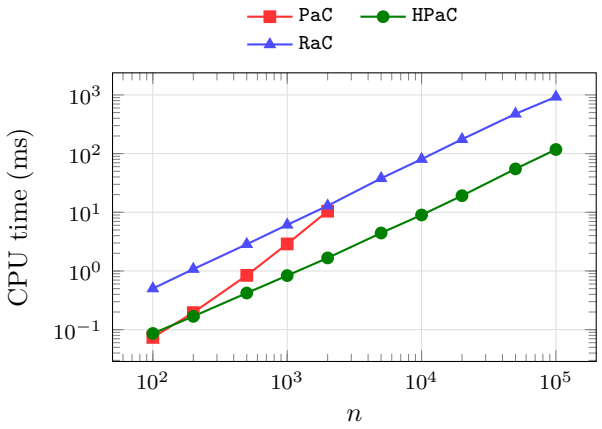

Figure 3: Median CPU time on path-mixed instances (campaign D, 60 instances per size).

the reasons of Section 7.2 – but its size is only partly accounted for by the heap policy: a direct comparison of the two heap implementations on these instances shows a factor of 1.3–1.4, not 3–4. The remainder is attributable to the code being an independent rewrite rather than a refactoring, and to instances regenerated under different coefficient families. We therefore treat absolute times across the two versions as incomparable, and only trends as comparable.

7.6 Comparison with parametric pseudoflow

BPPF is the unmodified upstream implementation of the parametric pseudoflow method, applicable to arbitrary precedence graphs. It evaluates minimum cuts at user-supplied parameter values, so a comparison requires deciding what to supply, and we drive it in the setting most favourable to it, identical to the methodology of the manuscript's first version: for each instance, one single BPPF process receives, in BPPF's native affine (two-numbers-per-arc) capacity format, the $k+1$ parameter values that bracket all k breakpoints. BPPF is thus spared the search for the breakpoints, and one call certifies the entire parametric solution. Its own internal solve timer (input parsing excluded) is compared against HPaC's in-process time for the full sweep on the same instance; the fixed-point precision is 10^{-6}.

On all 2 400 instances with $n \leq 1\,000$, the median paired ratio BPPF/HPaC grows from 1.5 at $n = 100$ to 4.5 at $n = 1\,000$, with an overall median of 3.3. Two qualifications belong with this number. First, it is measured on forests with $n \leq 1\,000$, and BPPF solves a strictly more general problem, so it says nothing about BPPF outside this class. Second, the closures returned by the two methods agree at every probe on every instance, with one exception class: on 16 of the 2 400 instances BPPF merges two consecutive closure layers whose thresholds differ by less than its fixed-point precision, and therefore does not recover the exact optimal layer sequence. All 16 belong to

	CPU time (ms)			
n	PaC	HPaC	DHPaC	RaC
100	0.1	0.1	0.1	0.4
200	0.2	0.2	0.2	0.9
500	0.9	0.6	0.7	2.3
1 000	3.1	1.3	1.4	4.9
2 000	11.2	2.6	2.7	10.1
5 000	–	6.8	7.2	27.3
10 000	–	14.3	15.0	57.5
20 000	–	32.0	32.1	123.1
50 000	–	92.3	92.5	315.6
100 000	–	205.6	211.1	663.7

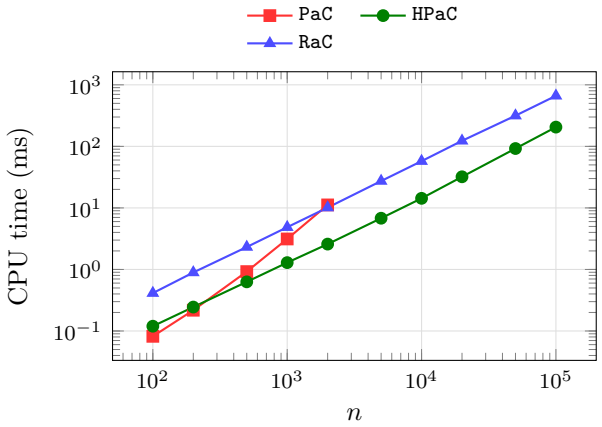

Figure 4: Median CPU time on `binary-mixed` instances (campaign D, 60 instances per size).

	CPU time (ms)			
n	PaC	HPaC	DHPaC	RaC
100	0.1	0.6	0.6	0.4
200	0.2	2.0	2.0	0.8
500	1.1	12.6	13.2	2.1
1 000	4.5	62.0	62.0	5.2
2 000	17.6	231.2	226.1	9.7
5 000	123.7	1 427.5	1 429.4	24.7
10 000	643.1	6 898.7	6 479.8	63.3
20 000	2 890.8	26 127.1	24 619.1	122.1
50 000	19 336.1	–	–	291.7
100 000	77 541.1	–	–	606.8

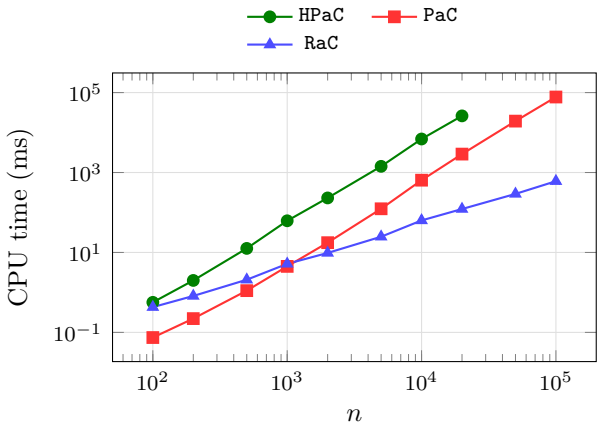

Figure 5: Median CPU time on `star-mixed` instances (campaign D, 60 instances per size). The heap-based algorithms stop at the preregistered cutoff $n = 20\,000$; `PaC` and `RaC` run the full range.

`near-ties` and `exact-ties`, the families designed precisely to place thresholds at that distance. The algorithms of this study are unaffected, since they compare integers exactly; this is the practical consequence of exact arithmetic, and the reason the comparison reports agreement separately from timing.

An earlier, discarded variant of this comparison drove `BPPF` once per breakpoint, one process spawn per call. That measurement is reported nowhere: the resulting ratio is dominated by process-creation overhead and says nothing about either algorithm.

8 Computational conclusions

Correctness is uniform across the study: 16 200 instances, 292 080 timed runs, seven algorithms, zero cross-algorithm disagreements, on top of the exhaustive enumeration oracle on small instances and the sanitizer builds. No run hit the time or memory limit, so no result is censored or imputed.

On the instances tested, `HPaC` is the algorithm of choice on mixed-orientation random forests, paths and balanced binary trees, faster than `RaC` by a factor between 2 and 13 depending on size and density, and faster than the direct scan from $n \approx 400$ on. `RaC` is the algorithm of choice on stars, where its advantage grows with n and reaches about 250× at $n = 20\,000$ – the one class where its

Table 13: Stars, paired ratio `RaC/HPaC` by family: values below 1 favour `RaC`.

family	#inst	median `RaC/HPaC`	IQR
`anti-correlated`	80	0.060	0.227
`correlated`	80	0.061	0.238
`exact-ties`	80	0.060	0.214
`independent-positive`	80	0.060	0.203
`independent-signed`	80	0.062	0.212
`near-ties`	80	0.064	0.218

Table 14: Stars, paired ratio `RaC/PaC` by size: the crossover between the two non-heap-limited algorithms.

n_nodes	Paired instances	Median rac/pac	IQR
100	60	5.778	0.454
200	60	3.675	0.192
500	60	1.883	0.120
1000	60	1.151	0.053
2000	60	0.524	0.034
5000	60	0.198	0.016
10000	60	0.100	0.005
20000	60	0.043	0.004
50000	60	0.015	0.001
100000	60	0.007	0.000

	CPU time (ms)			
n	`HPaC`	`DHPaC`	`HIPaC`	`RaC`
100	0.1	0.1	0.1	0.3
200	0.2	0.2	0.1	0.6
300	0.3	0.3	0.2	0.9
400	0.4	0.4	0.2	1.2
500	0.4	0.5	0.3	1.4
600	0.5	0.6	0.3	1.7
700	0.7	0.8	0.4	2.0
800	0.7	0.8	0.4	2.3
900	0.8	1.0	0.5	2.4
1 000	1.1	1.0	0.6	3.1
10 000	14.2	14.1	6.5	45.9
20 000	29.5	31.1	14.4	120.3
30 000	49.6	51.0	25.0	231.3
40 000	73.8	72.9	35.2	347.9
50 000	95.3	101.9	45.1	444.2
60 000	109.4	131.0	52.8	570.2
70 000	127.8	151.0	66.0	730.2
80 000	168.4	185.0	78.7	980.0
90 000	203.1	219.7	86.5	1 184.7
100 000	237.6	201.7	114.2	1 620.5

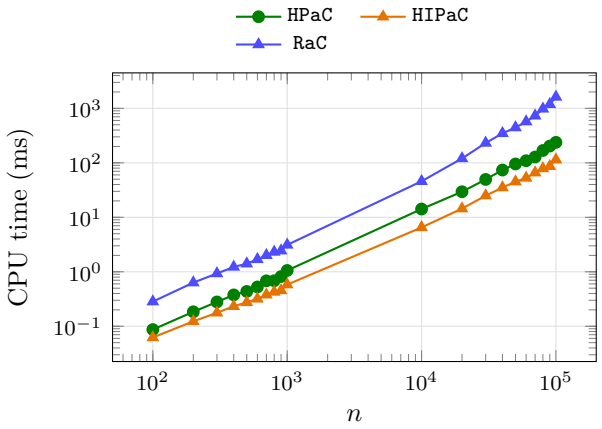

Figure 6: Median CPU time on `in-forest` instances (campaign E, 240 instances per size).

better worst-case bound is visible in practice, for a structural reason that also explains why the direct scan resists there better than any heap-based schedule. `HIPaC` and `HOPaC` deliver a stable 1.3–1.9× speedup over the general algorithm when the orientation is known in advance. Against a general parametric pseudoflow implementation on forests with $n \leq 1\,000$, `HPaC` is faster by a median factor of 3.3 even when the pseudoflow solver is handed the breakpoint locations, and unlike it recovers the exact layer sequence on every instance.

Two structural facts organize all of the above. The topology decides which algorithm wins – density moves the margin by an order of magnitude and the star class inverts the ordering entirely – while the coefficient family decides only how much parametric structure exists, moving absolute times but never the ranking. And the choice of data structure inside one algorithm can matter as much as the choice of algorithm: the two heap policies of Section 7.2 differ by 4× in time and two orders of magnitude in memory, at identical output and identical asymptotic bounds.

None of these statements is a claim of general dominance: each is scoped to the topologies, sizes and coefficient families in which it was measured, and the star class is the standing reminder that the ordering can invert completely when the topology changes.

	CPU time (ms)			
n	HPaC	DHPaC	HOPaC	RaC
100	0.1	0.1	0.1	0.3
200	0.2	0.2	0.1	0.6
300	0.3	0.3	0.2	0.9
400	0.4	0.4	0.3	1.2
500	0.6	0.5	0.3	1.6
600	0.6	0.7	0.4	1.7
700	0.7	0.8	0.5	2.4
800	0.8	0.9	0.5	2.3
900	0.9	1.0	0.6	2.5
1 000	1.1	1.3	0.7	3.1
10 000	13.7	15.3	7.6	44.1
20 000	31.9	33.7	16.5	118.0
30 000	51.4	53.4	27.8	217.4
40 000	73.6	79.8	40.3	323.3
50 000	95.3	100.6	54.7	463.8
60 000	106.1	135.1	61.5	541.2
70 000	135.6	155.2	75.5	722.2
80 000	164.4	197.6	88.4	941.6
90 000	185.7	220.6	106.0	1 187.5
100 000	229.5	287.2	132.1	1 608.8

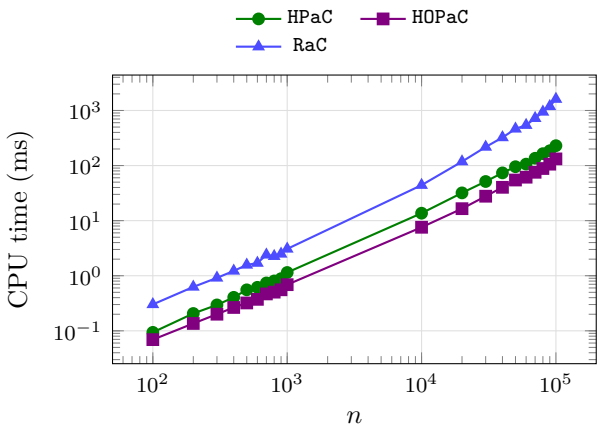

Figure 7: Median CPU time on `out-forest` instances (campaign E, 240 instances per size).

Table 15: In-forests, `HIPaC`/`HPaC` by family.

family	#inst	median `HIPaC`/`HPaC`	IQR
`anti-correlated`	800	0.689	0.705
`correlated`	800	0.697	0.741
`exact-ties`	800	0.671	0.617
`independent-positive`	800	0.676	0.710
`independent-signed`	800	0.652	0.659
`near-ties`	800	0.653	0.620

Table 16: Out-forests, `HOPaC`/`HPaC` by family.

family	#inst	median `HOPaC`/`HPaC`	IQR
`anti-correlated`	800	0.716	0.659
`correlated`	800	0.732	0.638
`exact-ties`	800	0.657	0.582
`independent-positive`	800	0.743	0.768
`independent-signed`	800	0.726	0.653
`near-ties`	800	0.711	0.615

Code and data availability

The code, instance generators, raw and processed benchmark data and the analysis pipeline are available at https://github.com/fabiofurini/parametric-closure-forests, release `v0.3.0`. The release attaches the full instance archive (split under GitHub's asset size limit) and the raw and processed benchmark data, with SHA-256 checksums. Every table and every plot in this report is generated from `results/raw/*.csv` by `tools/build_report.sh` and `tools/emit_report_tables.py`; regenerating them from the same raw data reproduces this report exactly. The pseudoflow baseline is the unmodified upstream implementation available at https://github.com/hochbaumGroup/Bounded-precision-simple-parametric.git.